Architecture Review Summary

> Generated: 2026-03-22
> Scope: full codebase (`src/`)

Contents

1. Layer Dependency Check
   - Violation 1 — audiodataplane.h (L1 → L4)
   - Violation 2 — audiostreamcontroller.h (L3 → L4)
   - Violation 3 — audiocontroller_interface.h (L3 → L4)
   - Violation 4 — audiocontroller.cpp (L3 → L4 × 2)
   - Violation 5 — audiostreamcontroller.cpp (L3 → L4)
   - Violation 6 — sample.h (L2 → L4)
   - CMake-Level Violations
2. Real-Time Safety
   - Issue 1 — resize() in RT callback path
   - Issue 2 — LOG_INFO in MIDI RT callback
   - Issue 3 — dynamic_pointer_cast in RT statistics path
3. Observations
   - Structural Concerns
   - Deprecated Components
   - Missing Test Coverage
4. Diagrams
   - Diagram A — Component Architecture
   - Diagram B — Track Hierarchy
   - Diagram C — Interface Hierarchy

Layer Dependency Check

Layer model:

Layer	Directory	Namespace
0	`src/framework/`	`miniaudioengine::core`
1	`src/data/`	`miniaudioengine::core`
2	`src/processing/`	`miniaudioengine::audio`
3	`src/control/`	`miniaudioengine::audio` / `::midi`
4	`src/public/` + `include/miniaudioengine/`	`miniaudioengine`

Files reviewed: 49 (headers + sources across all layers)

Layer	Files	Violations

Layer	Files	Violations
0 — Framework	12	0
1 — Data	8	1
2 — Processing	6	1
3 — Control	11	4
4 — Public	14	0
Total	**51**	**6**

> **Violations found: 7 include-level violations across 6 source files, plus 3 CMake link-time violations.**

Violation 1

```
[VIOLATION] src/data/audio/include/audiodataplane.h (Layer 1)
            includes "miniaudioengine/filehandle.h" (Layer 4)
```

`AudioDataPlane` stores a `FileHandlePtr` and calls `preload_wav_file(const FileHandlePtr&)`. `filehandle.h` lives in `include/miniaudioengine/` which is the Layer 4 public API boundary.
Fix: Introduce a Layer 0/1 abstract file-source interface (e.g. `IFileSource`) and have the data plane depend only on that interface; inject the concrete `FileHandle` from the control layer at setup time.

Violation 2

```
[VIOLATION] src/control/audio/include/audiostreamcontroller.h (Layer 3)
            includes "miniaudioengine/audiodevice.h" (Layer 4)
```

`AudioStreamController` exposes `std::vector<core::IAudioDevicePtr> get_audio_devices()` which constructs `AudioDevice` objects (the L4 concrete type, now also deprecated).
Fix: Return `std::vector<core::IAudioDevicePtr>` or the newer `std::vector<DeviceHandlePtr>` and keep the concrete `AudioDevice` construction inside the `.cpp` file, or eliminate `AudioDevice` in favour of `DeviceHandle` entirely.

Violation 3

```
[VIOLATION] src/control/audio/include/audiocontroller_interface.h (Layer 3)
            includes "devicehandle_factory.h" (Layer 4,
src/public/io/devicemanager/include/)
```

`IAudioController` uses `DeviceHandlePtr` and `DeviceHandleFactory` in its interface. `devicehandle_factory.h` is in the `devicemanager` module (`src/public/io/`), which is Layer 4.
Fix: Forward-declare `DeviceHandle` at Layer 0 or Layer 3 and move the factory call into the `.cpp` file. Alternatively, relocate `DeviceHandle`/`DeviceHandleFactory` to a shared Layer 0 or Layer 1 header since they are simple value types used across layers.

Violation 4

```
[VIOLATION] src/control/audio/src/audiocontroller.cpp (Layer 3)
            includes "miniaudioengine/trackmanager.h" (Layer 4)

[VIOLATION] src/control/audio/src/audiocontroller.cpp (Layer 3)
            includes "devicehandle_factory.h" (Layer 4)
```

`AudioController::_start()` calls `register_dataplanes()` which walks `TrackManager` to gather dataplanes. This creates a direct dependency from the control layer upward to the public API.
Fix: Pass dataplanes into the controller at `start()` time via parameter or pre-registration (i.e. invert control — the public layer calls `register_dataplane()` before calling `start()`).

Violation 5

```
[VIOLATION] src/control/audio/src/audiostreamcontroller.cpp (Layer 3)
            includes "miniaudioengine/trackmanager.h" (Layer 4)
```

Same `TrackManager` pull-through pattern as Violation 4, present in the newer `AudioStreamController` implementation.
Fix: Same as Violation 4.

Violation 6

```
[VIOLATION] src/processing/audio/include/sample.h (Layer 2)
            includes "miniaudioengine/wavfile.h" (Layer 4)
```

`Sample` is constructed from a `WavFilePtr`, which is the deprecated Layer 4 concrete type. `wavfile.h` is now a deprecated stub following the PImpl refactor.
Fix: Refactor `Sample` to accept the new `FileHandlePtr` (or a plain `IFileSource` interface). Remove the `wavfile.h` dependency entirely once `Sample` is updated.

CMake-Level Violations

The CMakeLists link graphs contain additional upward dependencies not expressible as `#include` violations but still structurally unsound. These propagate all transitive headers into the lower layer's include path.

```
[VIOLATION-CMAKE] data-audio (Layer 1) links PUBLIC: trackmanager (Layer 4)
[VIOLATION-CMAKE] data-audio (Layer 1) links PUBLIC: devicemanager (Layer 4)
[VIOLATION-CMAKE] control-audio (Layer 3) links PUBLIC: devicemanager (Layer 4)
```

The `trackmanager` link on `data-audio` is the most serious: it makes the entire public API visible from within the real-time data layer at compile time.

Real-Time Safety

Files reviewed: 8 (`src/data/audio/` × 4, `src/data/midi/` × 4)
Issues found: 3

Issue 1

```
[RT-ALLOC] src/data/audio/src/audiodataplane.cpp:179
           m_output_buffer.resize(buffer_size, 0.0f)
           inside AudioDataPlane::prepare_output_buffer()
           called directly from AudioDataPlane::process_audio() — the RtAudio RT
callback
```

`prepare_output_buffer` unconditionally resizes `m_output_buffer` when its size doesn't match `n_frames * m_output_channels`. This can trigger a heap allocation inside the audio callback on the first call or whenever channel count changes.
Fix: Pre-allocate `m_output_buffer` to the maximum expected size in `_start()` (where allocation is permitted) and use `std::fill` only in the callback. Remove the size check from the RT path.

Issue 2

```
[RT-BLOCK] src/data/midi/src/mididataplane.cpp:17
           LOG_INFO("MidiDataPlane received MIDI message: " +
midi_message.to_string())
           inside MidiDataPlane::process_midi_message()
           called from MidiCallbackHandler::midi_callback() — the RtMidi RT
callback
```

`LOG_INFO` is a logging macro that typically acquires a lock, formats a string (heap allocation), and writes to stdout or a file — all blocking operations. String concatenation (`+ midi_message.to_string()`) also allocates.

Fix: Remove logging from the RT callback. Use `LockfreeRingBuffer` to post the `MidiMessage` to the control thread and log there.

Issue 3

```
[RT-ALLOC] src/data/audio/src/audiodataplane.cpp (lines ~130-145)
           std::dynamic_pointer_cast<AudioOutputStatistics>(p_statistics)
           inside AudioDataPlane::update_audio_output_statistics()
           called from AudioDataPlane::process_audio() — the RtAudio RT callback

[RT-ALLOC] src/data/midi/src/mididataplane.cpp (lines ~26-34)
           std::dynamic_pointer_cast<MidiInputStatistics>(p_statistics)
           inside MidiDataPlane::update_midi_input_statistics()
           called from MidiDataPlane::process_midi_message() — the RtMidi RT
callback
```

`std::dynamic_pointer_cast` increments/decrements atomic reference counts and may take shared_ptr internal locks on some implementations. More critically, the derived statistics update writes to a `shared_ptr`-managed object from the RT thread while the control thread might read the same object without synchronisation.
Fix: Use a plain `std::atomic<uint64_t>` or a lock-free stats struct with `atomic` members instead of a `shared_ptr<IDataPlaneStatistics>`. Apply a `static_cast` instead of `dynamic_pointer_cast` if the type is known at construction time.

Observations

Structural Concerns

1. **Control layer pulls `TrackManager` to discover dataplanes.** `AudioController` and `AudioStreamController` both `#include "miniaudioengine/trackmanager.h"` and call `TrackManager::instance()` inside `_start()` to walk registered tracks and gather `AudioDataPlane` pointers. This inverts the intended dependency: the public layer should configure the control layer before delegating to it. The pattern where every `start()` creates a coupling to the singleton registry makes the control layer untestable without a live `TrackManager`.

2. **`DeviceHandleFactory` at wrong layer.** After the PImpl refactor, `devicehandle_factory.h` was placed in `src/public/io/devicemanager/include/` (Layer 4). It is included directly from `src/control/audio/` (Layer 3), creating an upward dependency. `DeviceHandle` is a plain value type and `DeviceHandleFactory` is a thin factory — both should live at Layer 0 or be injected, not pulled from a higher layer.

3. **Circular dependency in CMake** — `data-audio` links `trackmanager` and `devicemanager`, while `trackmanager` links `data-audio`. This is a true circular dependency at the CMake target level (even if the header-level cycle is broken by forward declarations). This prevents incremental linker optimisation and makes isolated testing harder.

4. **Two parallel audio controller implementations.** `AudioController` (`src/control/audio/src/audiocontroller.cpp`) and `AudioStreamController` (`audiostreamcontroller.cpp`) coexist under `src/control/audio/`. Both include `trackmanager.h` and perform `_start()` implementations. It is unclear which is authoritative. The copilot instructions note `IAudioController` as deprecated, but `AudioStreamController` still uses the `AudioDevice` deprecated type and still includes `trackmanager.h`.

5. **`prepare_output_buffer` called unconditionally on every callback.** The RT callback calls `prepare_output_buffer(n_frames)` on the first line, which includes a conditional `resize`. Even when the condition is false, the conditional branch adds latency unpredictability. Consider validating once at stream-open time.

Deprecated Components

Component	Status	Still Used By
`AudioDevice` / `AudioDevicePtr`	Deprecated (PImpl refactor)	`audiostreamcontroller.h`, `audiostreamcontroller.cpp`, unit tests
`MidiDevice` / `MidiDevicePtr`	Deprecated (PImpl refactor)	Unit tests
`WavFile` / `WavFilePtr`	Deprecated (PImpl refactor)	`src/processing/audio/include/sample.h`, unit tests
`MidiFile` / `MidiFilePtr`	Deprecated stub	Unit tests
`IAudioController`	Deprecated (noted in docs)	`AudioStreamController` (base class), `TrackManager`
`include/miniaudioengine/audiodevice.h`	Deprecated stub header	`audiostreamcontroller.h`
`include/miniaudioengine/wavfile.h`	Deprecated stub header	`sample.h`

Missing Test Coverage

Component	Layer	Reason
`src/processing/audio/audioprocessor.cpp`	2	No test file exists for `IAudioProcessor`
`src/processing/audio/sample.h`	2	No test file exists for `Sample`

Component	Layer	Reason
`src/processing/sampleplayer/sampleplayer.cpp`	2	No test file exists for `SamplePlayer`
`src/public/cli/cli.cpp`	4	No test file exists for `CLI`
`src/control/audio/audiocontroller.cpp`	3	`AudioController` is only exercised transitively via `AudioStreamController` tests; direct unit tests are absent

Diagrams

Diagram A — Component Architecture

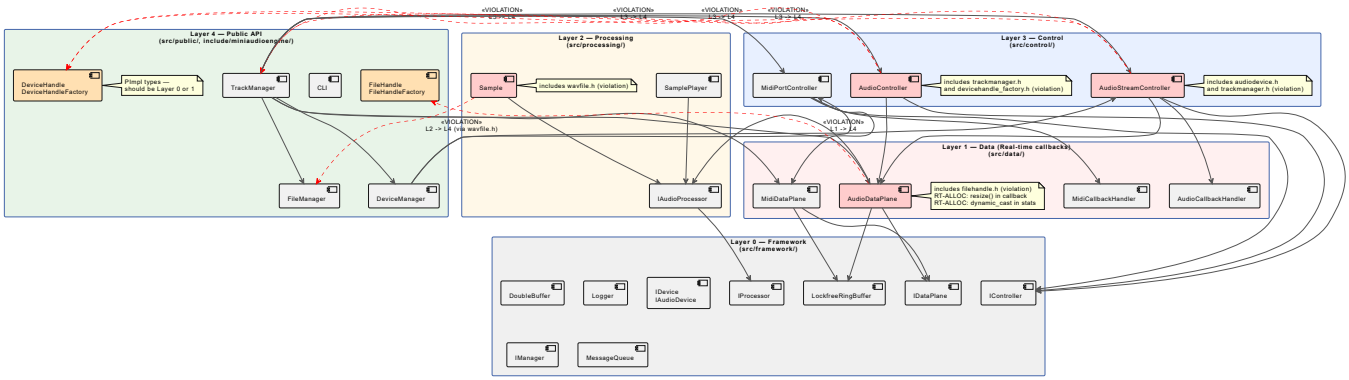

Diagram B — Track Hierarchy

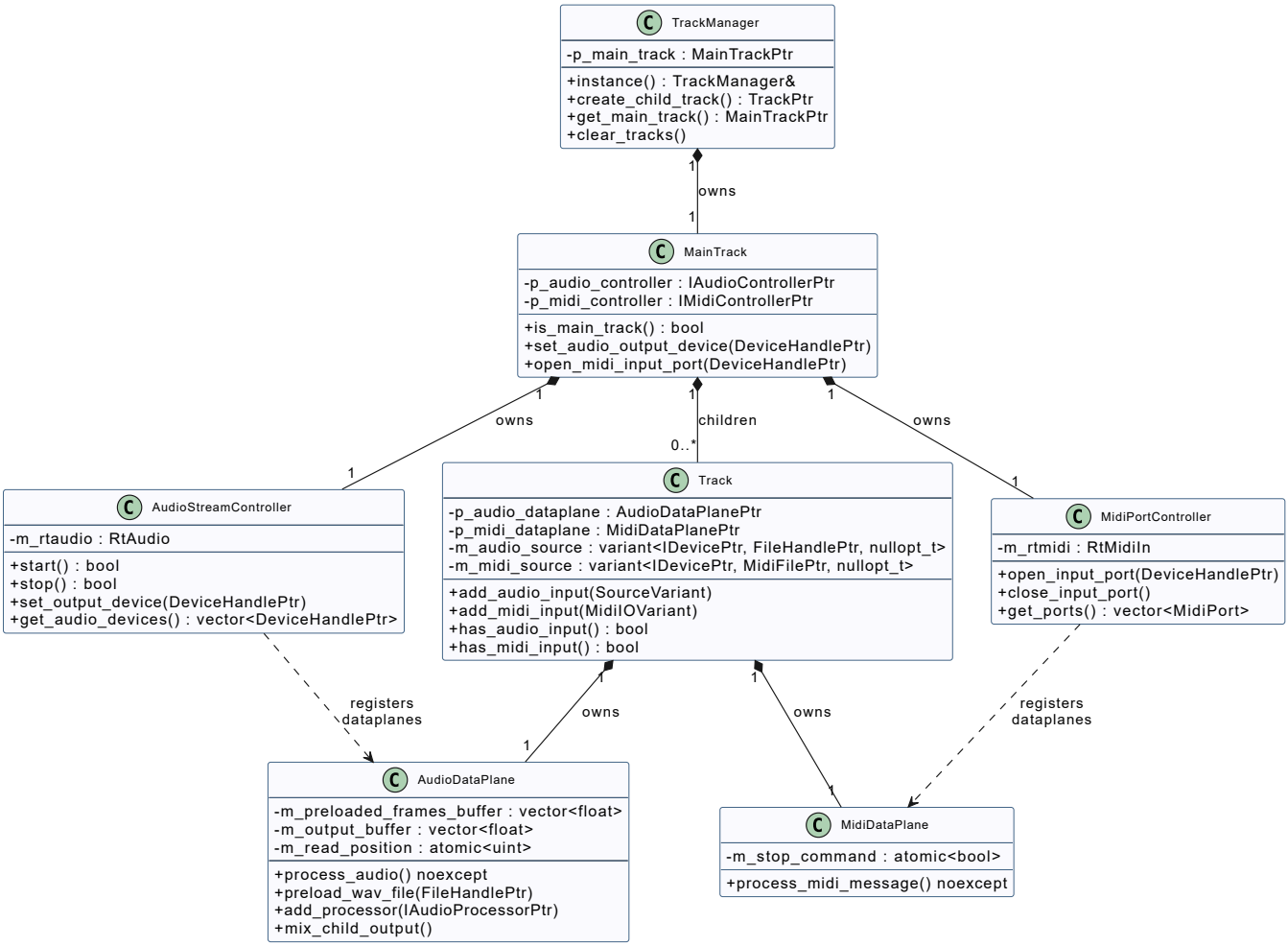
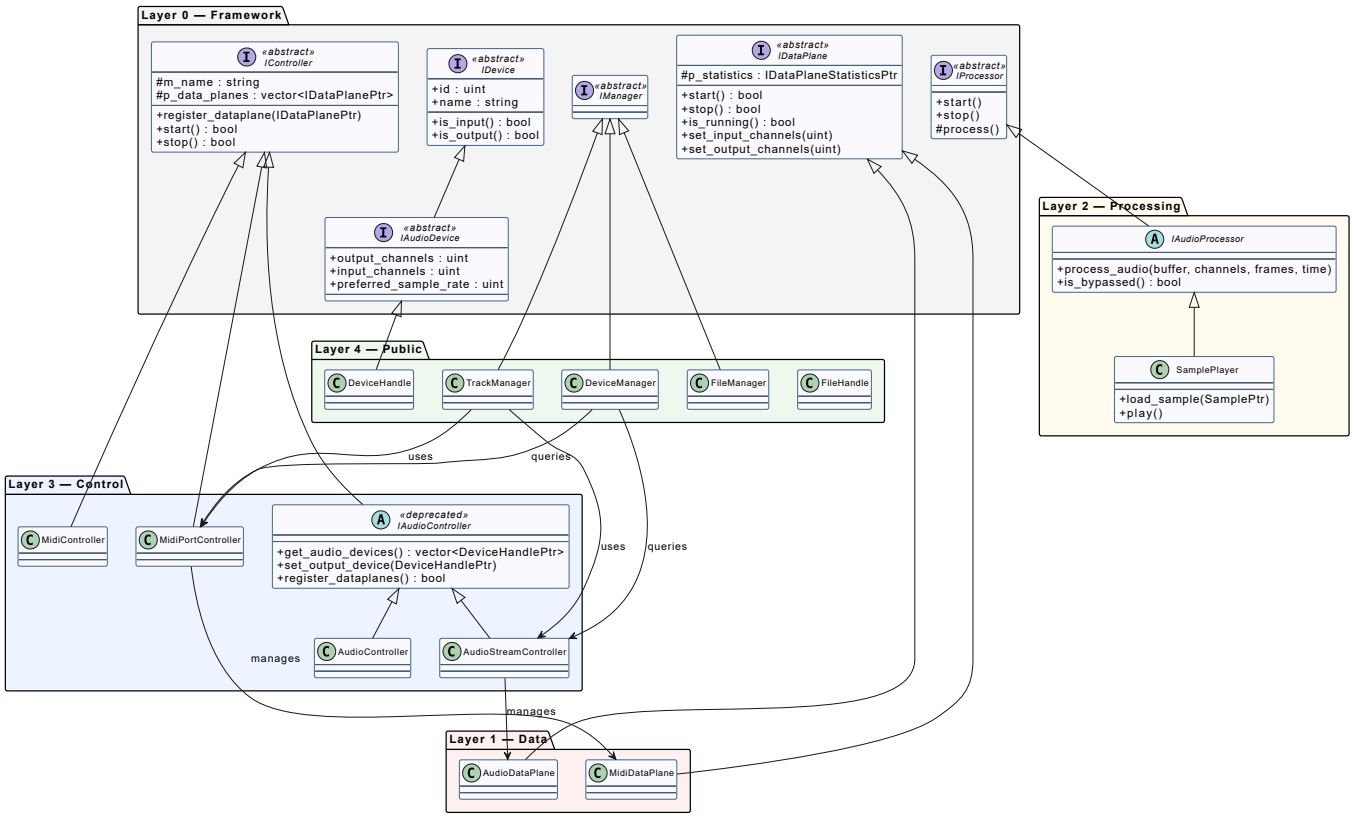
Diagram C — Interface Hierarchy